assignment

February 17, 2026

0.1 RDD Qsns

[1]:
```
from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("ASSIGNMENT").getOrCreate()
sc=spark.sparkContext
```

[ ]:
```
# 1. Clean customer_central.csv
"""
Customer Central.csv is having data quality issues, write an PySpark Core␣
 ↪program to clean the data based on below requirements.
Remove rows with any missing values for columns.
Replace multiple white spaces present in between customer name field.
Remove trailing and leading spaces from the field customer name, city and state.
Remove the punctuation characters present in postal code field.
Save the cleaned data in customer_central_cleaned.csv in HDFS for later usage.
"""
import re

df = sc.textFile("Customer_Central.csv")
header = df.first()
df_1 = df.filter(lambda x: x != header)

df_split = df_1.map(lambda x: x.split(","))

df_check = df_split.filter(lambda row: len(row) == 8)


def clean(row):
    c_id = row[0].strip()
    c_name = re.sub(r"\s+", " ", row[1].strip())
    segment = row[2].strip()
    country = row[3].strip()
    city = re.sub(r"\s+", " ", row[4].strip())
    state = row[5].strip()
    postal_code = re.sub(r"[^\w]", "", row[6])
    region = row[7].strip()
```

```
    return [c_id, c_name, segment, country, city, state, postal_code, region]


cleaned = df_check.map(clean)
data = cleaned.filter(lambda row: all(field.strip() != "" for field in row))

final_rdd = sc.parallelize([header]).union(data.map(lambda x: ",".join(x)))

# now save this as textfile
final_rdd.saveAsTextFile("customer_central_cleaned.csv") # this is divided into␣
 ↪multiple files(multiple partitions)

# to want a single file(use coalesce(1)) means it forces spark to use only 1␣
 ↪partition
# final_rdd.coalesce(1).saveAsTextFile("customer_central_cleaned.csv")
```

[ ]:
```
['C001,John Smith,Chicago,Illinois,60601123,Central,Consumer',
 'C002,Mary Johnson,Detroit,Michigan,48201,Central,Corporate',
 'C003,Robert Brown,Columbus,Ohio,43004,Central,Home Office',
 'C005,Patricia Davis,Milwaukee,Wisconsin,53202,Central,Corporate',
 'C007,Linda Wilson,Kansas City,Missouri,64101,Central,Home Office',
 'C008,William Moore,Minneapolis,Minnesota,55401,Central,Consumer',
 'C009,Barbara Taylor,Omaha,Nebraska,68102,Central,Corporate',
 'C010,James Anderson,Des Moines,Iowa,50301,Central,Consumer',
 'C011,Elizabeth Thomas,Wichita,Kansas,67202,Central,Consumer',
 'C012,David Jackson,Fargo,North Dakota,58102,Central,Corporate']
```

[ ]:
```
"""
2. Display total number of customers from each region.
Save the combined customer details in Customers.csv for later usage.

so display all regions,union all files with customer_central
"""

central_final=sc.textFile("customer_central_cleaned.csv")
west = sc.textFile("customer_west.csv")
east = sc.textFile("customer_east.txt")
south = sc.textFile("customer_south.csv")

# join others
header=east.first()

east=east.filter(lambda row:row!=header)
west=west.filter(lambda row:row!=header)
south=south.filter(lambda row:row!=header)

all_customers=central_final.union(east).union(west).union(south)
```

```
# save as single file
all_customers.coalesce(1).saveAsTextFile("Customers.csv")

"""
2. Display total number of customers from each region.
"""
customers = sc.textFile("Customers.csv")
split_data = customers.map(lambda row: row.split(","))
region_pair= split_data.map(lambda row:(row[7],1))
region_count = region_pair.reduceByKey(lambda a, b: a + b)
region_count.collect()
```

[ ]:
```
"""
3. Display Unique States in Alphabetical Order
"""
state_data = split_data.map(lambda row: row[5])
unique_states = state_data.distinct()
sorted_states = unique_states.sortBy(lambda state: state)
sorted_states.collect()
```

[ ]:
```
"""
4.Total Number of Unique Cities from Each State (Descending)
"""
#  (state, city)
state_city = split_data.map(lambda row: (row[5], row[4]))
# Remove duplicate (state, city)
unique_state_city = state_city.distinct()
# Convert to (state,1)
state_city_count = unique_state_city.map(lambda row: (row[0], 1))
# Count cities per state
state_city_total = state_city_count.reduceByKey(lambda a, b: a + b)
# Sort descending
sorted_state_city = state_city_total.sortBy(lambda row: row[1], ascending=False)

sorted_state_city.collect()
```

[ ]:
```
"""
5.display  First Name, Last Name, Segment, State (Florida & Virginia)
"""
# Filter Florida and Virginia customers
filtered_customers = split_data.filter(
    lambda row: row[5] == "Florida" or row[5] == "Virginia"
)

# Select required columns
selected_columns = filtered_customers.map(
```

```
    lambda row: (
        row[1].split(" ")[0],
        row[1].split(" ")[1],
        row[2],
        row[5]
    )
)

selected_columns.collect()
```

[ ]:
```
"""
6. Total Sales for Each Region (Increasing Order)
"""
orders = sc.textFile("order_details.csv")

orders_header = orders.first()

orders_data = orders.filter(lambda row: row != orders_header)

orders_split = orders_data.map(lambda row: row.split(","))

# Create (customer_id, sales)
order_pair = orders_split.map(lambda row: (row[1], float(row[2])))

# Customer data → (customer_id, region)
customer_pair = split_data.map(lambda row: (row[0], row[6]))

# Join orders with customers
joined_data = order_pair.join(customer_pair)

# joined_data → (customer_id, (sales, region))

# Convert to (region, sales)
region_sales = joined_data.map(lambda row: (row[1][1], row[1][0]))

# Aggregate total sales per region
total_region_sales = region_sales.reduceByKey(lambda a, b: a + b)

# Sort increasing
sorted_region_sales = total_region_sales.sortBy(lambda row: row[1],␣
 ↪ascending=True)

sorted_region_sales.collect()
```

0.2 DataFrames

[ ]:
```
from pyspark.sql.functions import col,sum,avg
```

[ ]:
```
"""
1. Display the total number of unique sub-categories under each category in the␣
 ↪alphabetical order of category.
"""
df = spark.read.csv("Product_Data.csv", header=True, inferSchema=True)

order_df = spark.read.csv("OrderDetails.csv", header=True, inferSchema=True)

res1 = (
    df.select("Category", "Sub-Category")
    .distinct()
    .groupBy("Category")
    .count()
    .orderBy("Category")
)
res1.show()
```

[ ]:
```
"""
2. Display the sub-categories which are under loss.
"""
joined = order_df.join(df, "Product ID", "inner")

loss_sub = (
    joined.groupBy("Sub-Category")
    .agg(sum("Profit").alias("TotalProfit"))
    .filter(col("TotalProfit") < 0)
)

loss_sub.show()
```

[ ]:
```
"""
3. Display the top 5 sub-categories with least sales.
"""
res3 = (
    joined.groupBy("Sub-Category")
    .agg(sum("Sales").alias("total_sales"))
    .orderBy("total_sales")
    .limit(5)
)
res3.show()
```

[ ]:
```
"""
4. Display the top 5 sub-categories with highest profit.
"""
```

```
res4 = (
    joined.groupBy("Sub-Category")
    .agg(sum("Profit").alias("total_profit"))
    .orderBy(col("total_profit").desc())
    .limit(5)
)

res4.show()
```

[ ]:
```
"""
5. Display average sales and average profit for each category in the decreasing␣
 ↪order of average sales.
"""
res5 = (
    joined.groupBy("Category")
    .agg(avg("Sales").alias("avg_sales"), avg("Profit").alias("avg_profit"))
    .orderBy(col("avg_sales").desc())
)

res5.show()
```

[ ]:
```
"""
6. Display the total sales done by customers from state "California" whose ship␣
 ↪mode is "Second Class" or "Same Day" in alphabetical order of customer name.
"""

cust_df = spark.read.csv("Customers1.csv", header=True, inferSchema=True)
joined_states = order_df.join(cust_df, "Customer ID", "inner")
res_6 = (
    joined_states.filter(
        (col("State").isin("California", "Georgia"))
        & (col("Ship Mode").isin("Second Class", "Same Day"))
    )
    .groupBy("State")
    .agg(sum("Sales").alias("total_sales"))
    .orderBy("State")
)
res_6.show()
```

0.3 SQL

If column name contains spaces, use backticks in spark SQL

Ex: select Customer Name from customers –> wrong fails

select `Customer Name` from customers

best practice is to rename columns in df

[ ]:
```
orders = spark.read.csv("OrderDetails.csv", header=True, inferSchema=True)
products = spark.read.csv("Product_Data.csv", header=True, inferSchema=True)
customers = spark.read.csv("Customers.csv", header=True, inferSchema=True)
```

[ ]:
```
# Create temporary views
orders.createOrReplaceTempView("orders")
products.createOrReplaceTempView("products")
customers.createOrReplaceTempView("customers")
```

[ ]:
```
"""
1. Display name, total sales and number of sales done by customers who placed␣
 ↪orders up to 3 times.
"""

spark.sql(
    """
SELECT
    c.`Customer Name`,
    SUM(o.Sales) AS total_sales,
    COUNT(o.`Order ID`) AS number_of_orders
FROM orders o
JOIN customers c
ON o.`Customer ID` = c.`Customer ID`
GROUP BY c.`Customer Name`
HAVING COUNT(o.`Order ID`) <= 3
"""
).show()
```

[ ]:
```
"""
2. Display name and total sales of customers with top 10 sales.
"""
spark.sql(
    """
SELECT
    c.`Customer Name`,
    SUM(o.Sales) AS total_sales
FROM orders o
JOIN customers c
ON o.`Customer ID` = c.`Customer ID`
GROUP BY c.`Customer Name`
ORDER BY total_sales DESC
LIMIT 10
"""
).show()
```

[ ]:
```
"""
3. Display number of sales in each product category.
```

```
"""
spark.sql(
    """
SELECT
    p.Category,
    COUNT(o.`Order ID`) AS number_of_sales
FROM orders o
JOIN products p
ON o.`Product ID` = p.`Product ID`
GROUP BY p.Category
"""
).show()
```

[ ]:
```
"""
4. Display total profit for each year in chronological order of year.
"""
spark.sql(
    """
SELECT
    YEAR(`Order Date`) AS year,
    SUM(Profit) AS total_profit
FROM orders
GROUP BY YEAR(`Order Date`)
ORDER BY year
"""
).show()
```

[ ]:
```
"""
5. Display total sales for each quarter in chronological order of quarter␣
 ↪(financial quarter).
"""
spark.sql(
    """
SELECT
    QUARTER(`Order Date`) AS quarter,
    SUM(Sales) AS total_sales
FROM orders
GROUP BY QUARTER(`Order Date`)
ORDER BY quarter
"""
).show()
```

[ ]:
```
"""
6. Display quarterly sales ordered by quarter (financial quarter) for each year.
"""
spark.sql(
    """
```

```
SELECT
    YEAR(`Order Date`) AS year,
    QUARTER(`Order Date`) AS quarter,
    SUM(Sales) AS total_sales
FROM orders
GROUP BY YEAR(`Order Date`), QUARTER(`Order Date`)
ORDER BY year, quarter
"""
).show()
```